INF201 Deliverable 3: Mini-RPG battle simulator

Repository: https://github.com/Trygve-Refsnes/git-deliverables

This notebook presents the six classes, checks their behavior, and compares complete battles with and without weapons. Run cells in order. A fixed random seed makes each weapon battle reproducible. The hero always attacks first, so battle order can affect the result.

In [1]
```
from character import Character, Warrior, Mage
from dice import Dice
from items import Weapon
from battle import Battle
import random
random.seed(201)
```

Character: health and encapsulation

Character stores name, hp_max, damage and an optional weapon. The protected _hp attribute is exposed through read-only hp and is_alive properties. take_damage(amount) and heal(amount) clamp health to the allowed range. attack(target) calls the target's take_damage method, allowing armor to work through polymorphism. introduce(), __str__() and __repr__() describe the character. Negative maximum HP is rejected.

In [2]
```
character = Character("Ada", hp_max=40, damage=8)
character.introduce()
print(str(character))
print(repr(character))
character.take_damage(100)
assert character.hp == 0 and not character.is_alive
character.heal(100)
assert character.hp == 40 and character.is_alive
for attribute in ("hp", "is_alive"):
    try:
        setattr(character, attribute, 0)
    except AttributeError:
        pass
    else:
        raise AssertionError(f"{attribute} must be read-only")
try:
    Character("Invalid", hp_max=-1)
except ValueError:
    print("Negative maximum HP rejected.")
else:
    raise AssertionError("Negative maximum HP accepted")
print("Health boundaries and read-only properties passed.")
Hi, my name is Ada!
Character: Ada with 40 HP and 8 damage
<Character name=Ada hp=40 damage=8>
Negative maximum HP rejected.
Health boundaries and read-only properties passed.
```

Dice: random damage

Dice has a sides attribute (default 6, minimum 2). roll() samples an integer from 1 through sides. __call__() lets d20() roll the same way as d20.roll(). __repr__() shows the configured sides.

In [3]
```
d6 = Dice()
```

```
d20 = Dice(20)
print(repr(d6), "d20 roll:", d20.roll(), "d20 call:", d20())
assert d6.sides == 6 and repr(d6) == "Dice(sides=6)"
assert all(1 <= d20() <= 20 for _ in range(100))
try:
    Dice(1)
except ValueError:
    print("Dice with fewer than two sides rejected.")
else:
    raise AssertionError("Invalid dice accepted")
```

```
Dice(sides=6) d20 roll: 3 d20 call: 11
Dice with fewer than two sides rejected.
```

Weapon: composition

Weapon stores name and damage_dice, a Dice object. roll_damage() delegates to that die. Character.attack() uses the equipped weapon instead of base damage. This composition lets weapons choose different dice without changing Character.

In [4]

```
sword = Weapon("Sword", Dice(8))

print(sword.name)
print(sword.damage_dice)
print(sword.roll_damage())
```

```
Sword
Dice(sides=8)
3
```

Warrior: defensive specialization

Warrior inherits Character attributes and methods and adds armor (default 5). Its take_damage() override subtracts armor from incoming damage, with a minimum of zero, before using the parent method. The checks below show both reduced and fully blocked damage.

In [5]

```
warrior = Warrior("Thor", armor=5)
warrior.take_damage(12)
assert warrior.hp == 93

warrior.take_damage(3)
assert warrior.hp == 93
```

Mage: mana and attacks

Mage inherits Character and adds mana and mana_max (both default 30). With at least 10 mana, attack() spends 10 and deals twice the base damage, ignoring weapons. Otherwise it uses Character.attack(), including an equipped weapon, and restores 5 mana up to mana_max. Initial mana is clamped between zero and mana_max.

In [6]

```
mage = Mage("Gandalf", mana=10, damage=10)
enemy = Character("Enemy")

mage.attack(enemy)
assert mage.mana == 0
assert enemy.hp == 80

mage.attack(enemy)
```

```
assert mage.mana == 5
assert enemy.hp == 70

# Test mana boundaries
mage2 = Mage("Merlin", mana=50, mana_max=30)
assert mage2.mana == 30

mage3 = Mage("Wizard", mana=-10)
assert mage3.mana == 0
```

Battle: turn-based combat

Battle stores hero and enemy. play_round() lets the hero attack first and permits a counterattack only if the enemy survives. simulate_battle(max_rounds=1000) repeats rounds until a character dies and prints the winner. It raises RuntimeError at the round limit to prevent endless combat, without treating one unlucky dice roll as a stalemate. A completed battle cannot attack again.

In [7]
```
hero = Character("Hero", hp_max=30, damage=10)
enemy = Character("Enemy", hp_max=20, damage=5)

battle = Battle(hero, enemy)
battle.simulate_battle()

assert hero.hp == 25
assert enemy.hp == 0
assert hero.is_alive is True
assert enemy.is_alive is False
Hero HP: 25, Enemy HP: 10
Hero HP: 25, Enemy HP: 0
Enemy has been defeated by Hero. YIPPEE!
```

Weapon behavior during Mage attacks

A fixed-damage test die isolates the rules from randomness. The first attack ignores the weapon and consumes mana; the next uses the weapon and restores mana.

In [8]
```
class FixedDice(Dice):
    def roll(self):
        return 7

staff = Weapon("Test staff", FixedDice(8))
mage = Mage("Tester", mana=10, damage=10, weapon=staff)
target = Character("Target")
mage.attack(target)
assert target.hp == 80 and mage.mana == 0
mage.attack(target)
assert target.hp == 73 and mage.mana == 5
print("Spell and weapon fallback checks passed.")
Spell and weapon fallback checks passed.
```

Complete battle without weapons: Warrior versus Mage

These characters rely on base damage and spells. Warrior armor reduces each incoming attack. Mage mana determines whether each turn uses a spell or a normal attack.

In [9]
```
unarmed_warrior = Warrior("Thor", hp_max=60, damage=12)
```

```
unarmed_mage = Mage("Gandalf", hp_max=60, damage=10)
Battle(unarmed_warrior, unarmed_mage).simulate_battle()
assert unarmed_warrior.is_alive != unarmed_mage.is_alive
print("Remaining mana:", unarmed_mage.mana)
Thor HP: 45, Gandalf HP: 48
Thor HP: 30, Gandalf HP: 36
Thor HP: 15, Gandalf HP: 24
Thor HP: 10, Gandalf HP: 12
Thor HP: 10, Gandalf HP: 0
Gandalf has been defeated by Thor. YIPPEE!
Remaining mana: 5
```

Complete battle with weapons: Warrior versus Mage

Fresh characters use the same health and base damage as above. The sword rolls a d12 and the staff rolls a d8. Weapon damage replaces base damage, so equipping a weapon can lower damage as well as make it variable. Mage spells still ignore the staff.

In [10]

```
random.seed(201)
armed_warrior = Warrior("Armed Thor", hp_max=60, damage=12,
                        weapon=Weapon("Sword", Dice(12)))
armed_mage = Mage("Armed Gandalf", hp_max=60, damage=10,
                  weapon=Weapon("Staff", Dice(8)))
Battle(armed_warrior, armed_mage).simulate_battle()
assert armed_warrior.is_alive != armed_mage.is_alive
print("Remaining mana:", armed_mage.mana)
Armed Thor HP: 45, Armed Gandalf HP: 58
Armed Thor HP: 30, Armed Gandalf HP: 46
Armed Thor HP: 15, Armed Gandalf HP: 40
Armed Thor HP: 15, Armed Gandalf HP: 33
Armed Thor HP: 14, Armed Gandalf HP: 28
Armed Thor HP: 0, Armed Gandalf HP: 16
Armed Thor has been defeated by Armed Gandalf. BUMMER!
Remaining mana: 0
```

Results and limitations

In the unarmed example, Thor wins with 10 HP and Gandalf has 5 mana left. With weapons and seed 201, Gandalf wins with 16 HP and no mana left. The sword can roll below Thor's base damage, while the staff also changes Gandalf's fallback attack. These examples show that weapons can change the outcome.

The printed rounds show the impact of armor, mana consumption and random weapon damage. Compare the surviving character and remaining HP in the two battles. These are illustrative examples rather than evidence that one class is always stronger: the hero attacks first, dice vary and different health or damage settings can change the winner. Health always remains within bounds for valid initial maxima. The round limit stops configurations where neither character can win; reaching it does not declare a winner.

In [11]

```
blocked_hero = Warrior("Shield A", damage=5)
blocked_enemy = Warrior("Shield B", damage=5)
try:
    Battle(blocked_hero, blocked_enemy).simulate_battle(max_rounds=3)
except RuntimeError as error:
    print(error)
else:
    raise AssertionError("Blocked battle should reach its round limit")
assert blocked_hero.hp == blocked_enemy.hp == 100
fallen = Character("Fallen", hp_max=0)
```

```
survivor = Character("Survivor")
Battle(fallen, survivor).play_round()
assert survivor.hp == 100
print("Round limit and finished-battle checks passed.")
Shield A HP: 100, Shield B HP: 100
Shield A HP: 100, Shield B HP: 100
Shield A HP: 100, Shield B HP: 100
Battle reached the round limit without a winner.
Round limit and finished-battle checks passed.
```

GenAI use

Codex compared the repository against the assignment PDF and supplied lecture simulator. It identified missing notebook explanations, battle demonstrations and submission artifacts, and flagged negative maximum HP and potentially endless battles. At the user's request it added validation and a battle round limit, expanded and executed this notebook, created the PDF exporter, added regression tests and submission documentation, and prepared Git changes for pushing. The checks confirm selected behavior, but do not replace understanding or review of the code. The group should review all changes, explain them in the video and supplement this disclosure with any earlier AI use; earlier use is not known from this session.